

Trabalho Prático (T)

Identificação da Disciplina

Código da Disciplina	Nome da Disciplina	Professor	Período
CIC_5MA e EGS_5MA	Programação Concorrente	Jeremias Moreira Gomes	2026/1

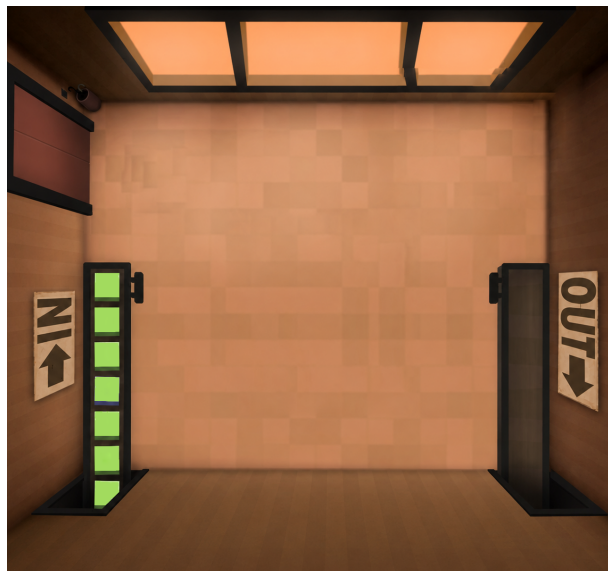
1. Objetivo

O objetivo deste trabalho é capacitar o aluno a estudar e compreender conceitos envolvendo programação concorrente. Utilizando o conhecimento adquirido nas aulas, o aluno deverá desenvolver um sistema cujo o gerenciamento e execução de tarefas envolvam a utilização de recursos que são utilizados por múltiplas *threads*.

Ao cumprir os objetivos do trabalho, o aluno terá adquirido: (i) uma compreensão prática dos conceitos e características-chave de programação concorrente; (ii) maior aprimoramento das suas habilidades de escrita técnica, organização de informações e comunicação eficaz, uma vez que a documentação de implementação exigirá uma apresentação clara e estruturada das principais características do sistema; e (iii) essa experiência proporcionará maior confiança e capacidade para explorar esses conceitos na solução de problemas computacionais.

2. Enunciado

A *Indústria de Deslocamento de Pacotes (IDP)* é uma empresa especializada em soluções para logística automatizada e movimentação inteligente de cargas em centros de distribuição. Seu principal objetivo é desenvolver sistemas capazes de transportar grandes quantidades de pacotes entre diferentes setores de uma fábrica, utilizando esteiras automatizadas e agentes autônomos de coleta e entrega.



Com o crescimento acelerado do comércio eletrônico e da demanda por processos logísticos cada vez mais eficientes, a IDP tem investido no desenvolvimento de robôs autônomos para movimentação interna de materiais. Esses robôs são responsáveis por coletar pacotes em áreas de armazenamento, transportá-los até pontos de processamento e realizar entregas em setores de expedição, reduzindo a necessidade de intervenção humana e aumentando a capacidade operacional dos centros de distribuição.

Um dos principais desafios encontrados nesses ambientes é garantir que pacotes sejam transportados de forma eficiente entre áreas de armazenamento e despacho, respeitando limitações físicas dos recursos disponíveis. Em um centro de distribuição real, diversos robôs operam simultaneamente coletando pacotes em pontos de armazenamento, posicionando-os em esteiras transportadoras e recebendo-os em setores de expedição. Como todos compartilham os mesmos recursos físicos, é necessário coordenar cuidadosamente suas ações para evitar colisões, bloqueios e perda de produtividade.

O funcionamento do sistema é relativamente simples. Pacotes são gerados continuamente em pontos de coleta distribuídos pela área de armazenamento. Robôs coletores devem localizar esses pacotes, transportá-los até a entrada da esteira e inseri-los para processamento. A esteira possui capacidade limitada e desloca os pacotes automaticamente em direção ao setor de expedição, avançando uma posição por unidade de tempo.

No setor de expedição, robôs entregadores são responsáveis por retirar os pacotes da saída da esteira e encaminhá-los para os pontos de despacho disponíveis. O sistema deve operar continuamente enquanto houver pacotes aguardando transporte ou processamento, respeitando as restrições de movimentação do ambiente e a capacidade limitada dos recursos compartilhados.

Assim, visando validar a próxima geração de sistemas logísticos da empresa, o seu *squad* foi designado para desenvolver uma simulação do centro de distribuição da IDP. A solução deverá permitir a configuração de diferentes cenários de operação, possibilitando avaliar o comportamento do sistema sob diferentes quantidades de robôs, capacidades de esteira e volumes de pacotes.

3. Centro de Distribuição Automatizado

Na simulação do centro de distribuição da IDP, todo o processo de transporte de pacotes ocorre de forma automatizada por meio de robôs móveis e uma esteira transportadora. O objetivo do sistema é coletar pacotes em áreas de armazenamento, transportá-los até a esteira e posteriormente encaminhá-los para os setores de despacho.

O ambiente é dividido em três regiões principais: a área de coleta, a esteira transportadora e a área de expedição. A Figura 1 apresenta um esquema simplificado da organização do sistema.

Na área de coleta existem estações geradoras de pacotes, representadas pelos pontos P . Essas estações produzem pacotes ao longo da execução da simulação, os quais permanecem aguardando coleta até que um robô coletor esteja disponível para transportá-los.

Os robôs coletores deslocam-se pela área de coleta buscando pacotes nas estações geradoras. Após retirar um pacote, o robô deve transportá-lo até a entrada da esteira, representada pela posição i_n . Cada robô pode transportar apenas um pacote por vez.

A esteira transportadora conecta a área de coleta à área de expedição. Ela é composta por uma sequência de posições discretas e possui capacidade limitada. Cada posição da esteira pode armazenar no máximo

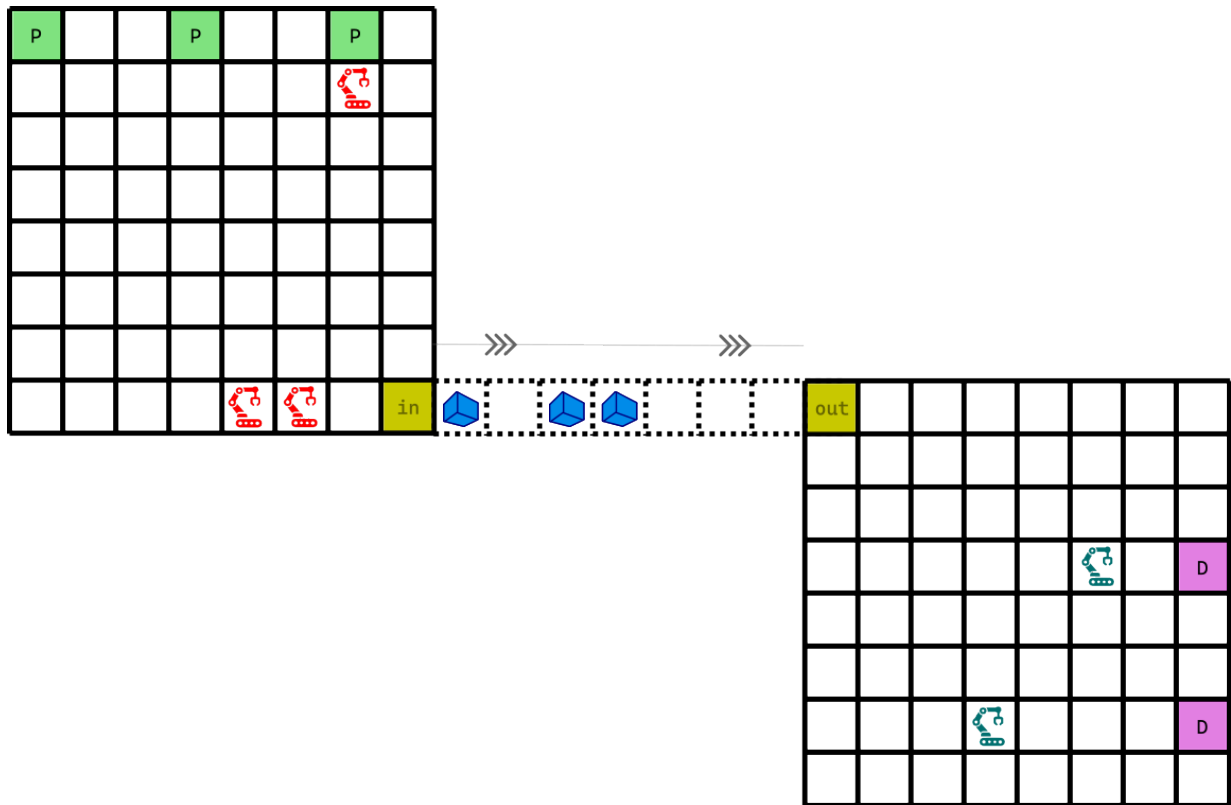


Figura 1. Esquema de Organização do Centro de Distribuição

um pacote. Uma vez inseridos na esteira, os pacotes deslocam-se automaticamente em direção à saída, representada pela posição *out*. A cada unidade de tempo, os pacotes avançam uma posição na direção da saída, desde que a próxima posição esteja livre.

Na área de expedição encontram-se os robôs entregadores. Esses robôs são responsáveis por retirar os pacotes da saída da esteira e transportá-los até os pontos de despacho, representados por *D*. Assim como os coletores, cada robô entregador pode transportar apenas um pacote por vez.

Como diversos robôs operam simultaneamente em um mesmo ambiente, todos os deslocamentos devem respeitar as restrições físicas do sistema. Em particular, **duas entidades não podem ocupar simultaneamente a mesma célula do mapa**. Dessa forma, a movimentação dos robôs, a ocupação das posições da esteira e o acesso às áreas de coleta e despacho devem ser coordenados de maneira a garantir a consistência do estado global da simulação.

O objetivo do sistema é transportar pacotes entre as áreas de coleta e expedição por meio dos robôs e da esteira transportadora. A simulação termina quando uma quantidade pré-definida de pacotes tiver sido entregue aos pontos de despacho ou quando uma condição de encerramento definida pelo aluno/grupo for atingida.

4. Implementação

A implementação do trabalho deve ser feita em C. A organização dos arquivos fica a cargo do aluno/grupo, o qual deverá conter instruções de compilação, bem como um arquivo `Makefile` para facilitar a compilação do código. Além disso, os testes e validação do trabalho serão feitos em um ambiente Linux, similar ao disponibilizado nos laboratórios da disciplina.

4.1. Elementos da Simulação

Como o objetivo do trabalho é proporcionar uma experiência prática com programação concorrente, a implementação deverá conter, no mínimo, os seguintes elementos, que funcionarão como *threads* ou recursos compartilhados do sistema:

- Gerador de Pacotes
- Interface
- Robôs Coletores
- Esteira Transportadora
- Robôs Entregadores

4.1.1. Gerador de Pacotes

Essa entidade é responsável por gerar novos pacotes durante a execução da simulação. Os pacotes devem surgir nas estações de coleta presentes na área de armazenamento (posições P do mapa).

A geração pode ocorrer em intervalos regulares ou aleatórios, ficando a critério do aluno/grupo definir a estratégia adotada. Cada pacote gerado deverá eventualmente ser coletado, transportado pela esteira e entregue a uma área de despacho.

As estações de coleta constituem recursos compartilhados do sistema, uma vez que múltiplos robôs podem tentar coletar pacotes da mesma estação simultaneamente.

4.2. Interface

A simulação deverá possuir uma interface para acompanhamento da execução em tempo real. Esta deverá representar visualmente os elementos da simulação, incluindo o mapa, os robôs, os pacotes, a esteira transportadora e o painel de informações da execução.

No mínimo, a interface deverá apresentar:

- A área de coleta;
- A área de expedição;
- A posição dos robôs coletores;
- A posição dos robôs entregadores;
- O estado atual da esteira;
- A quantidade de pacotes aguardando coleta;
- A quantidade de pacotes atualmente na esteira;
- A quantidade de pacotes entregues;

- O tempo de execução da simulação.

Como diversas entidades alteram simultaneamente o estado do sistema, a interface deverá acessar os dados compartilhados de forma sincronizada, garantindo a consistência das informações exibidas.

A interface deverá ser implementada utilizando a biblioteca `ncurses`, que permite a criação de interfaces de texto em terminais de forma mais amigável e interativa. O Apêndice A contém um guia rápido de utilização da biblioteca `ncurses`.

Alternativamente, grupos que desejarem implementar uma interface gráfica utilizando a biblioteca `raylib` poderão substituir a interface baseada em `ncurses` por uma implementação gráfica equivalente, o que dará uma bonificação de até **10%** na nota final, a critério do professor. O Apêndice B contém um guia rápido de utilização da biblioteca `raylib`.

4.2.1. Robôs Coletores

Os robôs coletores são responsáveis por retirar pacotes das estações de coleta e transportá-los até a entrada da esteira (`in`).

Cada robô pode transportar apenas um pacote por vez. Para coletar um pacote, o robô deve deslocar-se até a estação que o contém, retirá-lo e, posteriormente, deslocar-se até a entrada da esteira para realizar sua inserção.

Caso a posição de entrada da esteira esteja ocupada, o robô deverá aguardar até que exista espaço disponível para inserir o pacote.

Durante a movimentação, duas entidades não podem ocupar simultaneamente a mesma célula do mapa. Dessa forma, os deslocamentos dos robôs devem respeitar as restrições de ocupação do ambiente.

4.2.2. Esteira Transportadora

A esteira transportadora é o principal mecanismo de transferência de pacotes entre a área de coleta e a área de expedição.

A esteira possui tamanho limitado e é composta por posições discretas. Cada posição pode armazenar, no máximo, um único pacote.

A cada unidade de tempo, os pacotes devem avançar uma posição em direção à saída da esteira (`out`), desde que a posição seguinte esteja livre. Caso contrário, o pacote permanece na posição atual até que o avanço seja possível.

A esteira constitui um buffer limitado e um recurso compartilhado do sistema, devendo ser adequadamente sincronizada para evitar inconsistências durante a movimentação dos pacotes.

4.2.3. Robôs Entregadores

Os robôs entregadores são responsáveis por retirar pacotes da saída da esteira (out) e encaminhá-los para os pontos de despacho (D).

Cada robô pode transportar apenas um pacote por vez. Para realizar uma entrega, o robô deve deslocar-se até a saída da esteira, retirar o pacote disponível e transportá-lo até uma posição de despacho.

Após a entrega do pacote, o contador global de pacotes despachados deverá ser atualizado e o robô ficará novamente disponível para uma nova tarefa.

Assim como ocorre com os robôs coletores, duas entidades não podem ocupar simultaneamente a mesma célula da área de expedição.

4.3. Sumário

Ao final do trabalho, o aluno/grupo deverá entregar a implementação completa da simulação do Centro de Distribuição Automatizado da IDP.

5. Metodologia

Este trabalho será dividido em duas partes: (i) Implementação da Simulação; e (ii) Relatório Técnico.

5.1. Implementação da Simulação (80 pts)

Nesta etapa, os alunos deverão implementar a simulação proposta, utilizando conceitos de programação concorrente. A implementação deverá ser feita em C e deverá conter os elementos descritos na Seção de Implementação.

A simulação deverá permitir a execução em diferentes cenários. Para isso, o aluno/grupo poderá optar por uma das seguintes abordagens:

- **Configuração Estática:** os parâmetros da simulação são definidos durante a implementação, através de cenários previamente configurados;
- **Configuração Dinâmica:** os parâmetros da simulação são definidos pelo usuário no momento da execução do programa.

Exemplos de parâmetros que podem ser configurados incluem:

- Dimensões do mapa;
- Quantidade de robôs coletores;
- Quantidade de robôs entregadores;
- Quantidade de estações geradoras de pacotes;
- Quantidade de pontos de despacho;
- Comprimento da esteira transportadora;
- Quantidade total de pacotes da simulação;
- Quantidade de obstáculos presentes no ambiente.

A implementação utilizando configuração estática poderá atingir até **90% da pontuação desta etapa**, desde que disponibilize diferentes cenários de execução previamente configurados e atenda corretamente aos demais requisitos do trabalho.

Recomenda-se que implementações com configuração estática possuam, no mínimo, três cenários distintos de execução, que pode ser escolhido via parâmetro no início da execução do programa ou através de um menu de seleção na interface da simulação.

Recomenda-se (o professor recomenda fortemente) implementar primeiro uma versão estática e mais simples da simulação, para garantir que o funcionamento básico do sistema esteja correto, antes de adicionar a complexidade que possa comprometer a corretude da simulação (entenda-se perder pontos por besteira).

5.2. Documentação (20 pts)

Nesta etapa, deve-se documentar a implementação da simulação, descrevendo principalmente:

- A organização das *threads*;
- Os recursos compartilhados existentes;
- As seções críticas identificadas;
- Os mecanismos de sincronização utilizados;
- As decisões de projeto adotadas durante a implementação.

A documentação deve conter também um guia de utilização da aplicação, incluindo instruções de compilação e execução em ambiente Linux similar ao disponibilizado nos laboratórios da disciplina (Ubuntu 24.04).

A documentação deverá ser concisa e objetiva, não deixando de ser clara, contendo no máximo **quatro páginas**, seguindo o Modelos para Publicação de Artigos da Sociedade Brasileira de Computação ([link](#)).

5.3. Interface Gráfica com Raylib (Bônus de até 10 pts)

A implementação padrão do trabalho prevê a utilização da biblioteca `ncurses` para construção da interface da simulação.

Como atividade opcional, grupos que substituírem integralmente a interface baseada em `ncurses` por uma interface gráfica utilizando a biblioteca `raylib` poderão receber uma bonificação de até 10 pontos.

A concessão da pontuação adicional ficará condicionada ao correto funcionamento da simulação concorrente.

6. Detalhes Acerca do Trabalho

6.1. Restrições deste Trabalho

Esta atividade poderá ser realizada em até **três integrantes**, que deverão ser informados ao professor da disciplina até 22/06/2026 (segunda-feira). Caso os integrantes não sejam informados, será considerado que o aluno está realizando o trabalho de maneira individual. **Caso um grupo seja formado, e não informe os integrantes ao professor, ou ocorra uma alteração na composição do grupo sem a devida comunicação no prazo estipulado, o trabalho será penalizado em 20% da nota final.**

6.2. Datas Importantes

Esse trabalho poderá ser submetido a partir de 17/06/2026 (quarta-feira) - 12:00h até 04/07/2026 (sábado) - 23:55h.

6.3. Por onde será a entrega o Trabalho?

O trabalho deverá ser entregue via Ambiente Virtual (<https://ambientevirtual.idp.edu.br/>), na disciplina de Programação Concorrente. Na disciplina haverá uma atividade chamada “Trabalho Prático (T)”, para submissão do trabalho.

6.4. O que deverá ser entregue, referente ao trabalho?

Deverá ser entregue a documentação e os códigos produzidos pelos alunos, relacionado a resolução do trabalho.

6.5. Como entregar o trabalho?

A submissão das atividades deverá ser feita em um arquivo único comprimido do tipo zip (Zip archive data, at least v1.0 to extract), contendo documentação e um diretório com os códigos elaborados¹. Além disso, para garantir a integridade do conteúdo entregue, o nome do arquivo comprimido deverá possuir duas informações (além da extensão .zip):

- As matrículas dos alunos, separada por hífen.
- O *hash* md5 do arquivo e .zip.

Exemplo: 20001101-20010203-202813378-3b1b448e9a49cd6a91a1ad044e67fff2.zip

Para gerar o md5 do arquivo comprimido, utilize o comando `md5sum` do Linux e em seguida faça o renomeamento utilizando o *hash* coletado.

Segue um exemplo de geração do arquivo final a ser submetido no moodle, para um aluno:

```
[13930] j3r3mias@tardis:trabalho-01 > ls
trabalho-01-apc-jeremias.c
[13931] j3r3mias@tardis:trabalho-01 > zip -r aaa.zip trabalho-01-apc-jeremias.c
  adding: trabalho-01-apc-jeremias.c (deflated 99%)
[13932] j3r3mias@tardis:trabalho-01 > ls
aaa.zip  trabalho-01-apc-jeremias.c
[13933] j3r3mias@tardis:trabalho-01 > ls -lha aaa.zip
-rw-rw-r-- 1 j3r3mias j3r3mias 335 out 15 16:54 aaa.zip
[13934] j3r3mias@tardis:trabalho-01 > md5sum aaa.zip
44cf46f9237cb506ebde3e9e1cd044f9  aaa.zip
[13935] j3r3mias@tardis:trabalho-01 > mv aaa.zip 160068000-44cf46f9237cb506ebde3e9e1cd044f9.zip
[13936] j3r3mias@tardis:trabalho-01 > ls -lha 160068000-44cf46f9237cb506ebde3e9e1cd044f9.zip
-rw-rw-r-- 1 j3r3mias j3r3mias 335 out 15 16:54 160068000-44cf46f9237cb506ebde3e9e1cd044f9.zip
[13937] j3r3mias@tardis:trabalho-01 > md5sum 160068000-44cf46f9237cb506ebde3e9e1cd044f9.zip
44cf46f9237cb506ebde3e9e1cd044f9  160068000-44cf46f9237cb506ebde3e9e1cd044f9.zip
[13938] j3r3mias@tardis:trabalho-01 >
```

Figura 2. Exemplo de geração de arquivo para submissão

6.6. Quem deverá entregar o trabalho?

O Ambiente Virtual disponibiliza uma opção de trabalhos em grupo, onde um membro do grupo realiza a submissão e o arquivo é disponibilizado para ambos. Caso esta opção não esteja disponível, todos os alunos deverão submeter o trabalho no Ambiente Virtual onde, **somente um aluno deverá gerar a versão final do trabalho (arquivo zip)**, para que o *hash* do arquivo não corra o risco de ficar diferente prejudicando a avaliação da entrega.

¹ Evite enviar junto, arquivos desnecessários como diretórios `.git/`, por exemplo

6.7. Observações importantes

- **Não deixe para fazer o trabalho de última hora.**
- Não serão aceitos trabalhos com atraso.
- **Não deixe para fazer o trabalho de última hora.**
- Cópias de trabalho receberão zero (além disso, plágio é crime).
- **Não deixe para fazer o trabalho de última hora.**

6.8. Dicas de Implementação

Recomenda-se que o desenvolvimento da simulação seja realizado de forma incremental. Antes de introduzir mecanismos de concorrência, é aconselhável implementar uma versão simplificada do sistema contendo apenas a lógica básica e estruturas de dados.

Após validar o comportamento esperado da simulação, recomenda-se a introdução gradual das *threads* e mecanismos de sincronização. Essa abordagem tende a reduzir a complexidade da depuração, facilitando a identificação de problemas relacionados à concorrência.

Uma estratégia recomendada consiste em implementar e validar os seguintes componentes separadamente:

1. Representação do mapa e movimentação dos robôs;
2. Geração de pacotes nas estações de coleta;
3. Funcionamento da esteira transportadora;
4. Entrega de pacotes nos pontos de despacho;
5. Interface de acompanhamento da simulação;
6. Introdução das *threads* e mecanismos de sincronização.

Durante o desenvolvimento, é importante identificar os recursos compartilhados do sistema e planejar cuidadosamente o acesso a eles. Exemplos de recursos compartilhados incluem:

- As células do mapa;
- As estações geradoras de pacotes;
- As posições da esteira transportadora;
- Os pontos de despacho;
- Os contadores e estatísticas da simulação.

Recomenda-se também iniciar os testes com cenários simples e estáticos, para só depois evoluir para cenários mais complexos ou até dinâmicos.

Para a implementação da interface, recomenda-se a utilização da biblioteca `ncurses`, conforme descrito no Apêndice correspondente. Grupos que optarem pela implementação da interface gráfica utilizando `raylib` devem inicialmente validar o funcionamento da simulação antes de investir na construção de uma interface mais complexa.

7. Bibliografia

1. TANENBAUM, Andrew S. Sistemas Operacionais Modernos. Terceira edição. 2010.
2. `ncurses`: A Free Software emulation of `curses` in System V Release 6.6, and more. Disponível em: <https://invisible-island.net/ncurses/announce.html>.
3. Raylib: A Simple and Easy-to-use Library to Enjoy Game Programming. Disponível em: <https://www.raylib.com/>.

A. Ncurses: Guia Simplificado

A.1. Introdução

Ncurses é uma biblioteca de programação que permite a criação de interfaces gráficas em modo texto.

A.2. Instruções de Compilação

Utilize a flag `-lncurses` para compilar programas que utilizam a biblioteca `ncurses`. Exemplo:

```
gcc -o programa codigo.c -lncurses
```

A.3. Funções e Constante

- **LINES**: número de linhas da tela
- **COLS**: número de colunas da tela
- **initscr()**: inicializa o terminal em modo `curses`
- **endwin()**: finaliza o terminal que está em modo `curses`
- **clear()**: limpa a tela
- **echo()**: habilita a exibição do que é digitado
- **noecho()**: desabilita a exibição do que é digitado
- **keypad(stdsrc, TRUE)**: habilita o uso de teclas especiais
- **keypad(stdsrc, FALSE)**: desabilita o uso de teclas especiais
- **raw()**: desativa o buffer de quebra de linha (CTRL+Z e CTRL+C continuam funcionando)
- **noraw()**: ativa o buffer de quebra de linha
- **noraw()**: desabilita o modo `raw`
- **start_color()**: inicializa o uso de cores
 - **COLOR_BLACK**
 - **COLOR_RED**
 - **COLOR_GREEN**
 - **COLOR_YELLOW**
 - **COLOR_BLUE**
 - **COLOR_MAGENTA**
 - **COLOR_CYAN**
 - **COLOR_WHITE**
- **init_pair(número, cor_texto, cor_fundo)**: inicializa um par de cores
- **attron(atributos)**: liga um atributo
- **attroff(atributos)**: desliga um atributo
 - **A_NORMAL**: exibe o texto normalmente
 - **A_STANDOUT**: destaca o texto
 - **A_BOLD**: exibe o texto em negrito
 - **A_DIM**: exibe o texto com menos brilho
 - **A_UNDERLINE**: sublinha o texto
 - **A_REVERSE**: inverte as cores do texto e do fundo
 - **A_BLINK**: exibe o texto piscando
 - **A_ALTCHARSET**: conjunto de caracteres alternativos para os mesmos caracteres
- **printw("texto")**: exibe um texto na tela (utilizado no lugar de **printf**)

- **move(linha, coluna)**: move o cursor para a posição especificada
- **mvprintw(linha, coluna, "texto")**: exibe um texto na posição especificada (combinação de **move** e **printw**)
- **getch()**: lê um caractere digitado
- **getstr(string)**: lê uma string digitada
- **addch(caractere)**: exibe um caractere na tela na posição atual

A.4. Material de Apoio

O link <https://terminalroot.com.br/ncurses/> contém um Guia de Utilização da biblioteca ncurses, com exemplos de código e explicações mais detalhadas sobre várias funções.

A.5. Exemplo Simples de Código

```

1 // gcc -o programa exemplo-ncurses.c -lncurses
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <unistd.h>
5 #include <ncurses.h>
6
7 void imprime_conteudo(int tipo)
8 {
9     attron(COLOR_PAIR(7));
10    move(3, 0);
11    printf("Par_de_cores_%02d", tipo);
12    move(4, 0);
13    attron(COLOR_PAIR(tipo));
14    for (int i = '0'; i <= 'z'; i++) {
15        printf("%c", i);
16    }
17    attron(A_REVERSE);
18    move(5, 0);
19    attron(A_ALTCHARSET);
20    for (int i = '0'; i <= 'z'; i++) {
21        printf("%c", i);
22    }
23    attroff(A_ALTCHARSET);
24    attroff(A_REVERSE);
25    refresh();
26 }
27
28 int main()
29 {
30     int tecla;
31     initscr(); // Inicializa a tela (posição atual é (0, 0))
32     start_color();
33     raw(); // Não precisa esperar uma quebra de linha
34     noecho(); // O que for digitado não aparece na tela
35     keypad(stdscr, TRUE); // Teclas especiais como F1, F2, etc..
36
37     // Cria as cores que serão utilizadas (mas ainda não usa)
38     init_pair(7, COLOR_BLACK, COLOR_WHITE); // padrão
39     init_pair(1, COLOR_BLUE, COLOR_WHITE);
40     init_pair(2, COLOR_GREEN, COLOR_BLACK);
41     init_pair(5, COLOR_RED, COLOR_BLACK);
42
43     printf("Iniciando ...");
44     refresh(); // Realiza todos os comandos pendentes atualizando na tela
45     sleep(1);
46
47     attron(COLOR_PAIR(5));
48     mvprintw(LINES - 1, 0, "(%d_%d)_Pressione 'q' para sair ...", LINES, COLS);

```

```

49     attroff(COLOR_PAIR(5));
50
51     move(0, 0);
52     printw("_Digite 1, 2, ou F4:");
53     do {
54         tecla = getch();
55         switch (tecla) {
56             case '1':
57                 imprime_conteudo(1);
58                 break;
59             case '2':
60                 imprime_conteudo(2);
61                 break;
62             case KEY_F(4):
63                 attron(COLOR_PAIR(7));
64                 move(8, 0);
65                 printw("Tecla F4 foi pressionada .. saindo");
66                 refresh();
67                 sleep(1);
68             case 'q':
69             default:
70                 break;
71         }
72     } while (tecla != 'q' && tecla != KEY_F(4));
73
74     // Desabilitando tudo que foi habilitado antes de sair, para não deixar o
75     // terminal em estado diferente do anterior à execução
76     keypad(stdscr, FALSE);
77     noraw();
78     echo();
79
80     endwin();
81
82     printf("Termino da execucao\n");
83
84     return 0;
85 }

```

B. Raylib: Guia Simplificado

B.1. Introdução

Raylib é uma biblioteca multiplataforma para desenvolvimento de aplicações gráficas e jogos. Diferentemente do `ncurses`, que utiliza o terminal como interface, a Raylib permite criar janelas gráficas, desenhar formas, textos, imagens e animações em tempo real.

A utilização da Raylib é opcional neste trabalho e pode ser utilizada como alternativa à interface baseada em `ncurses`.

B.2. Instalação

O repositório oficial da Raylib contém instruções detalhadas para instalação em diferentes sistemas operacionais. No ambiente do laboratório Dell, utilizando o Ubuntu 24.04 do WSL, a instalação pode ser realizada da seguinte forma:

Instalação dos pré-requisitos:

```
sudo apt install build-essential git
sudo apt install libasound2-dev libx11-dev libxrandr-dev libxi-dev \
libgl1-mesa-dev libglu1-mesa-dev libxcursor-dev libxinerama-dev \
libwayland-dev libxkbcommon-dev
```

Para instalar a biblioteca Raylib, basta clonar o repositório oficial e compilar a biblioteca de maneira estática:

```
git clone --depth 1 https://github.com/raysan5/raylib.git raylib
cd raylib/src/
make PLATFORM=PLATFORM_DESKTOP
```

B.3. Compilação

Após a instalação da biblioteca, um programa utilizando Raylib pode ser compilado através do comando:

```
gcc -o prog codigo.c -lraylib -lm -lX11
```

B.4. Estrutura Básica de um Programa

Uma aplicação Raylib normalmente segue a seguinte estrutura:

1. Inicialização da janela;
2. Configuração da taxa de atualização;
3. Laço principal da aplicação;
4. Atualização da lógica do programa;
5. Desenho dos elementos da interface;
6. Encerramento da janela.

B.5. Funções Utilizadas no Exemplo

- **InitWindow(largura, altura, titulo)**: cria a janela da aplicação.
- **SetTargetFPS(valor)**: define a taxa de atualização desejada.
- **WindowShouldClose()**: verifica se o usuário solicitou o encerramento da aplicação.
- **BeginDrawing()**: inicia uma etapa de desenho.
- **EndDrawing()**: finaliza a etapa de desenho.
- **ClearBackground(cor)**: limpa a tela utilizando uma cor específica.
- **DrawRectangle(x, y, largura, altura, cor)**: desenha um retângulo preenchido.
- **DrawText(texto, x, y, tamanho, cor)**: desenha texto na tela.
- **CloseWindow()**: encerra a aplicação.

B.6. Entrada de Teclado

A Raylib possui diversas funções para leitura de teclado. Segue um exemplo de verificação se a seta para a direita foi pressionada:

- **IsKeyPressed(KEY_RIGHT)**: verifica se a seta para a direita foi pressionada.

B.7. Cores Disponíveis

A biblioteca possui diversas constantes de cores pré-definidas.

- BLACK
- WHITE
- RED
- GREEN
- BLUE
- YELLOW
- ORANGE
- PURPLE
- GRAY
- RAYWHITE

B.8. Representação de Mapas

Uma estratégia comum para representar ambientes bidimensionais consiste em utilizar matrizes.

No exemplo disponibilizado, cada posição da matriz representa uma célula do mapa:

- Valor 0: célula livre;
- Valor 1: parede.

O robô é representado por uma estrutura contendo sua posição no mapa, e o desenho é realizado convertendo coordenadas da matriz para coordenadas da janela gráfica.

B.9. Material de Apoio

O link <https://www.raylib.com/cheatsheet/cheatsheet.html> contém um Guia de Referência Rápida da biblioteca Raylib, com os headers, funções e constantes disponíveis de funcionalidades que podem ser utilizadas.

B.10. Exemplo Simples de Código

```

1  #include <raylib.h>
2  #define CELULAS_X 10
3  #define CELULAS_Y 10
4
5  typedef struct {
6      Vector2 posicao;
7  } Robo;
8
9  int main()
10 {
11     int largura = 500;
12     int altura = 500;
13
14     InitWindow(largura, altura, "Exemplo_de_mapa");
15
16     Robo robo = {0};
17     robo.posicao = (Vector2){1, 1};
18
19     float largura_da_celula = (float)largura / (float)CELULAS_X;
20     float altura_da_celula = (float)altura / (float)CELULAS_Y;
21
22     bool paredes[CELULAS_Y][CELULAS_X] = { false };
23
24     // Cuidado ao mexer nas dimensões do mapa, porque este foi criado (por
25     // preguiça já que é só um exemplo) pensando em algo (10, 10)
26     int map[CELULAS_Y][CELULAS_X] = {
27         {1, 1, 1, 1, 1, 1, 1, 1, 1, 1},
28         {1, 0, 0, 0, 0, 0, 0, 0, 0, 1},
29         {1, 0, 1, 1, 1, 1, 1, 1, 0, 1},
30         {1, 0, 0, 0, 1, 1, 0, 0, 0, 1},
31         {1, 0, 0, 0, 1, 1, 0, 0, 0, 1},
32         {1, 0, 0, 0, 1, 1, 0, 0, 0, 1},
33         {1, 0, 0, 0, 1, 1, 0, 0, 0, 1},
34         {1, 0, 1, 1, 1, 1, 1, 1, 0, 1},
35         {1, 0, 0, 0, 0, 0, 0, 0, 0, 1},
36         {1, 1, 1, 1, 1, 1, 1, 1, 1, 1}
37     };
38
39     for(int y = 0; y < CELULAS_Y; y++) {
40         for(int x = 0; x < CELULAS_X; x++) {
41             if(map[y][x] > 0) {
42                 paredes[y][x] = true;
43             }
44         }
45     }
46
47     SetTargetFPS(60);
48
49     while (!WindowShouldClose()) {
50         if (IsKeyPressed(KEY_RIGHT) && robo.posicao.x + 1 < CELULAS_X &&
51             paredes[(int)robo.posicao.y][(int)robo.posicao.x + 1] == false) {
52             robo.posicao.x+=1;
53         }
54         if (IsKeyPressed(KEY_LEFT) && robo.posicao.x - 1 > -1 &&
55             paredes[(int)robo.posicao.y][(int)robo.posicao.x - 1] == false) {
56             robo.posicao.x-=1;
57         }
58         if (IsKeyPressed(KEY_UP) && robo.posicao.y - 1 > -1 &&
59             paredes[(int)robo.posicao.y - 1][(int)robo.posicao.x] == false) {
60             robo.posicao.y-=1;
61         }
62         if (IsKeyPressed(KEY_DOWN) && robo.posicao.y + 1 < CELULAS_Y &&
63             paredes[(int)robo.posicao.y + 1][(int)robo.posicao.x] == false) {
64             robo.posicao.y+=1;
65         }

```



```

66
67 // Desenha tudo
68 BeginDrawing();
69
70     ClearBackground(RAYWHITE);
71
72     // Mapa
73     for(int y = 0; y < CELULAS_Y; y++){
74         for(int x = 0; x < CELULAS_X; x++){
75             if(map[y][x]==1){
76                 DrawRectangle(
77                     x * largura_da_celula ,
78                     y * altura_da_celula ,
79                     largura_da_celula ,
80                     altura_da_celula ,
81                     BLACK
82                 );
83             }
84         }
85     }
86     // Robô
87     DrawRectangle(
88         robo.posicao.x * largura_da_celula ,
89         robo.posicao.y * altura_da_celula ,
90         largura_da_celula ,
91         altura_da_celula ,
92         RED
93     );
94
95     DrawText("Use as setas do teclado", 0, 0, 20, WHITE);
96     EndDrawing();
97 }
98
99 CloseWindow();
100
101 return 0;
102 }

```